

"Methods Common to All Objects" Effective Java.

Ce chapitre traite des **méthodes héritées de** `java.lang.Object`, qu'on doit bien maîtriser et **souvent redéfinir** (`equals`, `hashCode`, `toString`, etc.).

→ Obey the general contract when overriding `equals`

Goal:

If you override `equals()`, you **must respect its contract**, otherwise your objects behave incorrectly in collections like `Set`, `Map`, etc.

When should you override `equals()` ?

Only if:

- Your class has **logical equality** (two objects are "equal" if their content is the same).
- Examples: `User`, `Point`, `Person`, `Email`.

Don't override if:

- Your class is meant for **unique instances only** (like singletons or utility classes).
- The class doesn't need logical equality (e.g. `Thread`, `DatabaseConnection`).

`equals()` must follow 5 rules (VERY IMPORTANT):

Rule	Meaning
Reflexive	<code>x.equals(x)</code> must be <code>true</code> .
Symmetric	If <code>x.equals(y)</code> is <code>true</code> , then <code>y.equals(x)</code> must also be <code>true</code> .
Transitive	If <code>x.equals(y)</code> and <code>y.equals(z)</code> are <code>true</code> , then <code>x.equals(z)</code> must also be <code>true</code> .
Consistent	Multiple calls should return the same result if nothing changes.
Non-nullity	<code>x.equals(null)</code> must be <code>false</code> (never throw

```
NullPointerException).
```

Bad equals() example:

```
1 public class Point {
2     public int x, y;
3
4     public boolean equals(Point other) { // ❌ not overriding
5         Object.equals!
6         return this.x == other.x && this.y == other.y;
7     }
8 }
```

This doesn't actually override `Object.equals(Object)` because the parameter isn't `Object`.

Correct `equals()` implementation:

```
1 @Override
2 public boolean equals(Object o) {
3     if (this == o) return true;
4     if (!(o instanceof Point)) return false;
5     Point p = (Point) o;
6     return x == p.x && y == p.y;
7 }
```

Explanation:

- `this == o`: check if they're the same object.
- `instanceof`: check if `o` is the right type.
- `(Point) o`: safe cast.
- Then compare the fields.

```
1 Point p1 = new Point(1, 2);
2 Point p2 = new Point(1, 2);
3
4 System.out.println(p1.equals(p2)); // true
```

But if you don't override `equals()`, this will print `false` — because `Object.equals()` only checks **reference equality** (same memory address).

Why does `equals()` return `false` if not overridden?

Because by default, `Object.equals()` is defined like this (simplified):

```
1 public boolean equals(Object obj) {
2     return this == obj;
3 }
```

What does `this == obj` mean?

It means: **Are both references pointing to the exact same object in memory?**

Always override hashCode when you override equals Y

Always override hashCode when you override equals

If you override `equals()` but **don't override** `hashCode()`, your objects will behave **incorrectly** in **hash-based collections** like:

- `HashMap`
- `HashSet`
- `Hashtable`

Problem example:

```
1 Point p1 = new Point(1, 2);
2 Point p2 = new Point(1, 2);
3
4 Set<Point> set = new HashSet<>();
5 set.add(p1);
6
7 System.out.println(set.contains(p2)); // ❌ false if hashCode() not
   overridden
```

Even though `p1.equals(p2)` is `true`, `HashSet` uses `hashCode()` to find the bucket, and if `p1` and `p2` have different hash codes, it won't find the match.

Contract between `equals()` and `hashCode()`:

If two objects are **equal** (via `equals()`), they **must** have the **same hash code**.

If two objects are **not equal**, they **can** still have the same hash code (but ideally not too often).

```
1 @Override
2 public int hashCode() {
3     return Objects.hash(x, y); // or custom formula
4 }
```

Now works correctly:

```
1 Set<Point> set = new HashSet<>();
2 set.add(new Point(1, 2));
3
4 System.out.println(set.contains(new Point(1, 2))); // true
```

When to override both `equals()` and `hashCode()`

You should override **both** when:

1. You care about **logical equality** (objects are equal if their contents are equal).
2. Your objects will be **stored in collections** like `HashSet`, `HashMap`, `Hashtable`, etc.
3. Your class is a **value object** (like `User`, `Point`, `Email`, `Product`, etc.)

Java collections like `HashMap` and `HashSet` use `hashCode()` **to find where to store or find your object**.

If you override `equals()` but not `hashCode()`, the collection may **not find your object** even if it's "equal" — because it looks in the **wrong hash bucket**.

Use `record` in Java 14+ → auto-generates both `equals()` and `hashCode()`.

Override only `equals()` when:

Rare — but possible when:

- You compare logically, but your object is **never used in a hash-based collection**.
- You're certain the object **won't be used as key** or in `Set`.

Not recommended unless you're 100% sure.

Why override `toString()`?

The default `toString()` method from `Object` looks like this:

```
1 | User@3e25a5
```

That's useless for logging, debugging, or printing your object.

If you override `toString()`, your object becomes **readable, debuggable**, and more **useful**.

Good `toString()` = easy to read

Example:

```
1 public class User {
2     private String name;
3     private String email;
4
5     @Override
6     public String toString() {
7         return "User{name='" + name + "', email='" + email + "'}";
8     }
}
```

```
9 }
```

Output:

```
1 | User{name='Houda', email='houda@gmail.com'}
```

Much better than: `User@1a2b3c4`

When is `toString()` used automatically?

- In `System.out.println(object)`
- In logs
- In debugger views
- In string concatenation:

```
"User is " + user
```

→ Behind the scenes, Java does this:

```
1 | System.out.println(user.toString());
```

Override `clone()` judiciously

- `clone()` is a method from `java.lang.Object`
- It creates a **copy of an object** in memory
- It's part of the `Cloneable` interface

But there's a problem...

The default `clone()` is **dangerous** and **confusing**:

- It makes a **shallow copy** (not deep)
- Throws `CloneNotSupportedException`
- Requires extra work to be done **correctly**
- Breaks easily in complex object graphs

That's why **Joshua Bloch** recommends avoiding `clone()` in most cases and using **copy constructors** or **static factory methods** instead.

OK, but when should I override `clone()` ?

Only if:

- You absolutely need a **performance-optimized copy**
- You have a **simple object**, or

- You're working with a **legacy system** that expects `clone()`

How to correctly override `clone()` :

1. Implement `Cloneable`
2. Override `clone()` and call `super.clone()`
3. Cast the result and copy mutable fields carefully

```
1 public class User implements Cloneable {
2     private String name;
3     private List<String> roles;
4
5     @Override
6     public User clone() {
7         try {
8             User cloned = (User) super.clone();
9             // Deep copy the mutable list
10            cloned.roles = new ArrayList<>(this.roles);
11            return cloned;
12        } catch (CloneNotSupportedException e) {
13            throw new AssertionError(); // should never happen
14        }
15    }
16 }
```

Bad clone (shallow copy):

```
1 cloned.roles = this.roles; // ❌ same list in both objects
```

→ Any change to `cloned.roles` will affect the original.

Better Alternative: Use a copy constructor or static factory

Copy constructor:

```
1 public User(User other) {
2     this.name = other.name;
3     this.roles = new ArrayList<>(other.roles);
4 }
```

Static factory:

```
1 public static User copyOf(User other) {
2     return new User(other);
3 }
```

Easier to maintain, works well with `final` fields, avoids `Cloneable` mess.

Consider implementing `Comparable`

`Comparable` is an **interface** that lets you define the **default (natural) sorting order** for your class.

sort `User` by name

```

1 public class User implements Comparable<User> {
2     private String name;
3
4     @Override
5     public int compareTo(User other) {
6         return this.name.compareTo(other.name);
7     }
8 }

```

How it works:

- `compareTo()` returns:
 - `< 0` if `this` is less than `other`
 - `0` if they're equal
 - `> 0` if `this` is greater than `other`

`compareTo()` contract (very important!)

Just like `equals()` and `hashCode()`, `compareTo()` must respect some rules:

Rule	Example
Consistent with equals	If <code>a.equals(b)</code> , then <code>a.compareTo(b) == 0</code>
Transitive	If <code>a > b</code> and <code>b > c</code> , then <code>a > c</code>
Anti-symmetric	If <code>a < b</code> , then <code>b > a</code>

Reason: `compareTo()` is a method on objects, not primitives

If `age` is declared as:

```

1 private int age; // primitive type

```

Then `this.age.compareTo(other.age)` will **not compile**, because:

- `int` is **not an object**
- It has **no methods**, including `compareTo`

So why `Integer.compare(...)` works?

Because `Integer.compare(int x, int y)` is a **static method** introduced in **Java 7+**, and works for primitives:

```

1 return Integer.compare(this.age, other.age); // safe for ints

```

Comparator

`Comparable` defines a **single “natural” order** (e.g., by name).

But sometimes, you want to sort your objects **in different ways**.

That's where `Comparator` comes in it allows **custom sorting logic** without modifying the class itself.

`Comparator` is an interface that lets you define **custom** sorting rules — even **multiple** orders.

It contains:

```
1 | int compare(T o1, T o2);
```

You don't modify the original class. You create the sort logic **outside** it.

How to use `Comparator`

Option 1: Use `Comparator.comparing(...)`

```
1 | List<User> users = ...
2 |
3 | // Sort by name
4 | users.sort(Comparator.comparing(User::getName));
5 |
6 | // Sort by age
7 | users.sort(Comparator.comparingInt(User::getAge));
8 |
```

Option 2: Chain Comparators

```
1 | // Sort by age, then by name
2 | users.sort(
3 |     Comparator.comparingInt(User::getAge)
4 |     .thenComparing(User::getName)
5 | );
```